

Functional Programming | FP

Summary

CONTENTS

1. Functional Language	3
1.1. Terms	3
1.2. Basic features of Functional Programming	3
1.3. Programming Paradigms	3
2. Haskell	4
2.1. Function definition	4
2.1.1. Curried functions	4
2.1.2. Guarded Equations	4
2.1.3. Pattern matching	4
2.1.4. List patterns	4
2.1.5. Lambda expressions	4
2.2. Function application	5
2.2.1. Point-free notation	5
2.3. Operators	5
2.4. Conditional expressions	5
2.5. List comprehension	5
2.6. Types	6
2.6.1. Type declarations	6
2.6.2. Data declarations	6
2.6.3. Polymorphism	8
2.6.4. Typeclasses	8
2.6.5. Newtype declarations	9
2.6.6. Instances	9
2.7. Modules	12
2.8. Lazy evaluation	13
2.9. Side effects	13
3. Lambda Calculus	13
3.1. Syntax	13
3.2. Definitions	14
3.2.1. nfin	14
3.2.2. substitution	14
3.2.3. α conversion	15
3.2.4. β reduction	15
3.2.5. δ reduction	15
3.2.6. η contraction	16
3.3. Evaluation	16
3.3.1. Relation to parameter passing	16
3.4. Encoding data and operations	16
3.5. Lambda cube	17
4. Proofs	18
4.1. Notation	18

4.1.1. Symbols	18
4.1.2. Proof tree	18
4.1.3. Two-Column Proof	19
4.1.4. Structured Proof	19
4.1.5. Sequent calculus	19
4.2. Formal logic	20
4.2.1. Formula	20
4.2.2. Proof	20
4.3. Equational reasoning	22
4.4. Induction	22
4.4.1. Logical inference	24
4.5. Theories	25
5. some cursed typescript ig	26

1. FUNCTIONAL LANGUAGE

Term	Definition
Functional programming	Style of programming with the focus on application of functions to arguments
Functional language	Languages that encourage functional programming

1.1. TERMS

Term	Definition
Mutual recursion	Two or more functions defined recursively in terms of each other (=calling each other)
Higher-order functions	A function is called higher-order if it takes a function as an argument or returns a function as a result.
Effectful programming	“Effectual” simply means that arguments and return values are no longer just plain (pure) values, but may also have so-called “effects” such as: – The possibility of failure, e.g. using the option type <code>Maybe</code> – Aggregating multiple results, e.g. using the list type <code>[]</code> – Performing IO, e.g. using the action type <code>IO</code>
Referential transparency	– no (mutable) variables – no assignments – no imperative control structures – all data structures are immutable

1.2. BASIC FEATURES OF FUNCTIONAL PROGRAMMING

- Referential transparency
- Functions as first-class citizens
- Higher-order functions
- Algebraic data types
- Pattern matching
- Recursion
- Types & type inference
- Haskell Specific: Type classes, Functors, Applicatives, Monads
- A Declarative Programming Paradigm.
- Foundation: Church’s Lambda calculus.
- Pure: No (or controlled) mutable state. Expressions are (by default) side effect free.

1.3. PROGRAMMING PARADIGMS

Paradigm	Definition
Imperative/Procedural	Normie
Object Oriented	Corpochud
Functional	Gigachad

Paradigm	Definition
Array	$Xy \leftarrow \text{array} \left[\begin{array}{c} \text{length} \\ \text{initial} \end{array} \right] \text{ of } \text{type} \text{ } a$ $F \leftarrow \lambda x. \text{if } x < 0 \text{ then } 1 \text{ else } (+/\div \div 3 + 1 \sim \times \tau +) Xy$ $\equiv F100 \div \uparrow 10$

2. HASKELL

2.1. FUNCTION DEFINITION

```
funName :: (TypeBound a) => a -> b           -- type definition
funName x = fnRequiringTypeBound x          -- implementation
```

2.1.1. Curried functions

```
add :: Int -> Int -> Int
add x y = x + y
```

```
add2 :: Int -> Int
add2 = add 2                                -- curried
```

2.1.2. Guarded Equations

```
abs n
  | n >= 0 = n                                -- fancy if else
  | otherwise = -n
```

2.1.3. Pattern matching

Patterns consist of variables and data constructors. They are matched in order, so more specific patterns should come first.

```
not :: Bool -> Bool
not False = True
not True = False
-- ==
not b = case b of
  True -> False
  False -> True
```

2.1.4. List patterns

```
head :: [a] -> a
head (x : _) = x                                -- first elem
tail :: [a] -> [a]
tail (_ : xs) = xs                             -- rest of list
```

2.1.5. Lambda expressions

```
-- \boundvar -> body
(\x -> x + x) 5
```

2.2. FUNCTION APPLICATION

```
-- function argument
sum [1..5]
```

2.2.1. Point-free notation

```
== Function composition
```

```
odd x = not (even x)
odd = not . even
```

The term “point-free” does not refer to the “.” character used for function composition (the number of which typically increase), refers to the fact that the definition does not mention the data points on which functions act.

Every definition has a point-free form that can be computed automatically! <http://pointfree.io>

2.3. OPERATORS

- Prefix notation is used for function application: `max 7 2`
- If infix notation is desired, one may use so-called operators (e.g. `+`): `7 + 2`
- Functions can be converted into operators using backticks: `7 `div` 2`
- Operators can be converted into functions using brackets: `(+) 7 2`

Notes:

- Operator symbols are formed from one or more symbol characters `! # $ % & * + . / < = > ? @ ^ | - ~ :` (or any Unicode symbol or punctuation).
- An operator symbol starting with `:` may only be used for data constructors (e.g. `:` is the constructor for lists).
- A so-called fixity declaration is used to specify the associativity and precedence level (1 (highest) to 9 (lowest)) of an operator. E.g.:
 - `infixl 6 +` specifies `(+)` to be left associative with level 6
 - `infixr 5 :` specifies `(:)` to be right associative with level 5
 - `infix 4 ==` specifies `(==)` to be neither left nor right associative (making parentheses mandatory while nesting), and with level 4
- Any operator lacking a fixity declaration is assumed to be `infixl 9`.
- Function application has a higher precedence than any infix operator.

2.4. CONDITIONAL EXPRESSIONS

Always ternary.

```
abs :: Int → Int
abs n = if n ≥ 0 then n else -n
```

2.5. LIST COMPREHENSION

$$\{x^2 \mid x \in \{1, 2, \dots, 5\}\}$$

$$\{x^2 \mid x \in \{1, 2, \dots, 12\}, 12 \bmod x \equiv 0\}$$

$$\{(x, y) \mid x \in \{1, 2\}, y \in \{4, 5\}\}$$

$$\{(x, y) \mid x \in \{1, 2\}, y \in \{x, \dots, 2\}\}$$

```
[x ^ 2 | x <- [1 .. 5]]           -- [1, 4, 9, 16, 25]
-- guards
[x | x <- [1 .. 12], 12 `mod` x == 0] -- [2, 3, 4, 6, 12]
-- multiple generators
[(x, y) | x <- [1, 2], y <- [4, 5]] -- [(1,4),(1,5),(2,4),(2,5)]
-- dependant generators
[(x, y) | x <- [1 .. 2], y <- [x .. 2]] -- [(1,1),(1,2),(2,2)]
```

2.6. TYPES

A Type in Haskell is a name for a collection of related values.

2.6.1. Type declarations

In Haskell, a new name for an existing type can be defined using a type declaration.

```
type String = [Char]
```

Type declarations can also have type parameters

```
type Pair a = (a, a)
```

```
mult :: Pair Int → Int
mult (m, n) = m * n
```

Type declarations can be nested but cannot be recursive

```
type Pos = (Int, Int)
type Trans = Pos → Pos      -- OK
```

```
type Tree = (Int, [Tree])   -- NOK
```

2.6.2. Data declarations

We can define new custom data types by declaring new types and functions that return values of these types. These functions (a.k.a. “data constructors” or “constructors”) are special since they cannot have a definition that results in a reduction rule. They therefore remain unchanged during execution and can be used to hold information.

```
data List a = Nil | Cons a (List a)
```

`List` has values of the form of `Nil` and `Cons` where `a` is a generic. `Nil` and `Cons` can be viewed as functions that construct values of type `List`.

A completely new type can be defined by specifying its values using a data declaration.

```
data Bool = False | True
```

- The two values `False` and `True` are called the constructors for the type `Bool`.
- Type and constructor names must always begin with an upper-case letter.
- Data declarations are similar to context free grammars. The former specifies the values of a type, the latter the sentences of a language.

Such data types are often referred to as algebraic datatypes.

- Type variables within types are implicitly universally quantified at the topmost level. For example, `a → Maybe a` actually denotes `forall {a}. a → Maybe a`. The `{}` brackets around `a` denotes that its instantiation can only be done automatically.
- You may ask `ghci` to explicitly print universal quantifiers as follows:

```

Prelude> :set -fprint-explicit-foralls
Prelude> :t Just
Just :: forall {a}. a -> Maybe a

```

- You may explicitly specify the universal quantification in polymorphic type signatures using the `ExplicitForAll` extension.

Maybe and Pair are called Type Constructors since they construct one type from another. This behaviour is expressed using kinds, which is the type of a “type”. The type system for kinds is very simple:

```
K ::= * | K -> K
```

* Kind of ordinary/proper/concrete/basic types

-> Kind of “type constructor” or “type operators”

- This is essentially the “simply typed lambda calculus” one level up, with one base type.
- Ordinary/proper/concrete/basic types are nothing more than nullary type constructors/operators. Analogy: A constant is nothing more than a nullary function.
- The word “type” is therefore often used for any type-level expression: ordinary/proper/concrete/basic types, and for type constructors/operators.

Examples: (Only types of kind * can have a value)

Kind	*	*	*	*	*	* -> *	* -> * -> *
Type	Bool	Char	[a] -> [a]	Maybe Char	a -> Maybe a	Maybe	(,)
Value	True	'a'	reverse	Just 'a'	Just		

(->) is also a type constructor!

Kind	*	*	* -> * -> *	* -> *	* -> *
Type	Bool -> Bool	(->) Bool Bool	(->)	(->) Bool	(->) a
Value	not	not			

The operator (->) is overloaded:

- In the context of a type (e.g. `id :: a -> a`), it denotes the type constructor for function types.
- In the context of a kind (e.g. `List :: * -> *`), it denotes the kind constructor for type constructors.

2.6.2.1. Records

Convenient syntax when data constructors have multiple parameters:

```

data Person = Person
  { name :: String
  , age  :: Int
  , children :: [Person]
  } deriving (Show)

```

-- ⇔

```

data Person = Person String Int [Person] -- positional constructor
  deriving (Show)
-- v selector functions v
name :: Person -> String
name (Person n _ _) = n

```

```
age :: Person → Int
age (Person _ a _) = a
```

```
children :: Person → [Person]
children (Person _ _ c) = c
```

Construction:

```
alice :: Person
alice = Person "Alice" 5 []
```

```
bob :: Person
bob = Person {name = "Bob", age = 35, children = [alice]}
```

Derived show contains labels and updates can use field labels:

```
-- >>> alice{age = age alice + 1}
-- Person {name = "Alice", age = 6, children = []}
```

2.6.3. Polymorphism

= “Many forms”

Term	Definition
Ad-hoc Polymorphism	Function with the same name denotes different implementations (function overloading, Interfaces)
Parametric Polymorphism	Code written to work with many possible types (OO: generics)
Subtype Polymorphism	One type (subtype) can be substituted for another (supertype)

2.6.4. Typeclasses

Haskell uses type classes to achieve ad-hoc polymorphism.

```
elem :: Eq a ⇒ a → [a] → Bool -- Eq is a typeclass
```

Type classes are declared using class declarations:

```
-- Eq = name of the declared type class
class Eq a where -- a = type parameter
    (==), (/=) :: a → a → Bool -- Required to be a member of Eq
    x /= y = not (x == y) -- Default implementation (optional)
```

Type classes may be extended:

```
-- Eq = name of the type class to be extended
class Eq a ⇒ Ord a where -- Ord = name of resulting type class
    (<), (≤), (≥), (>) :: a → a → Bool -- Additional requirements
```

- All members of the type class `Ord` are also members of the type class `Eq`.
- Members of `Ord` therefore also require `(=)` to be defined.

A type can be declared to be an instance of a type class using an instance declaration:

```
-- In order for (Maybe m) to be a member of the type class Eq, it is
-- required that the type m belongs to the type class Eq
instance Eq m ⇒ Eq (Maybe m) where
    Just x = Just y    = x == y
```

```
Nothing = Nothing = True
_       = _         = False
```

Just like it is possible to have higher-order functions, it is also possible to have higher-kinded types.

As far as the compiler is concerned, the only requirement for a type to be a member of a type class is that it should have a type correct instance declaration. There are of course additional semantic requirements that need to hold. These additional requirements are expressed as type class laws within the documentation of each type class.

Default implementations for some type classes can be generated automatically for data declarations using the deriving keyword:

```
data Bool = False | True
  deriving (Eq, Ord, Show, Read)
```

2.6.5. Newtype declarations

In cases where a datatype only has one constructor, a newtype declaration is used.

```
newtype ListOfBool = [Bool]
```

2.6.6. Instances

2.6.6.1. Functor

Functors are types that can be used to **wrap and extract values** of other types, and allow us to **lift unary functions** over the wrapped value.

Definition

```
class Functor f where
  fmap :: (a → b) → f a → f b
  (<$) :: a → f b → f a
  {-# MINIMAL fmap #-}
```

Type Class Laws

The fmap function **should not change the structure** of the Functor, **only its elements**.

An instance of Functor has to abide by the following laws:

- 1) fmap has to preserve identity
 - fmap id = id
- 2) fmap has to preserve function composition
 - fmap (g . h) = fmap g . fmap h

fmap

```
-- <$> = fmap
(<$>) :: Functor f => (a → b) → f a → f b
```

Examples

```
(+1) <$> Nothing      -- Nothing
(+1) <$> Just 1       -- Just 2
```

Instances

```
instance Functor Maybe where
  fmap _ Nothing = Nothing
  fmap f (Just x) = Just (f x)
```

2.6.6.2. Applicative

What if we want to lift an n-ary function?

Applicative Functors provide a **generic function to wrap values** (pure) and **generalized function application** (<*>).

Definition

```
class Functor f => Applicative f where
  pure  :: a -> f a
  (<*>) :: f (a -> b) -> f a -> f b
  liftA2 :: (a -> b -> c) -> f a -> f b -> f c
  (*>)  :: f a -> f b -> f b
  (<*)  :: f a -> f b -> f a
  {-# MINIMAL pure, ((<*>) | liftA2) #-}
```

Type Class Laws

- 1) Identity: pure has to preserve **identity**
– `pure id <*> x = x`
- 2) Homomorphism: pure has to preserve **function application**
– `pure (g x) = pure g <*> pure x`
- 3) Interchange: **Order of evaluation** must not matter when applying an effectful function to a pure argument
– `x <*> pure y = pure (\f -> f y) <*> x`
- 4) Composition: <*> has to be **associative**
– `x <*> (y <*> z) = (pure (.) <*> x <*> y) <*> z`

pure

We also need a function pure to lift 0-ary functions (constants):

```
pure :: a -> f a
f <$> x = (pure f) <*> x
```

Similarities to plain function application

```
($) :: (a -> b) -> a -> b
(<*>) :: f (a -> b) -> f a -> f b
```

```
(+) $ 11 $ 31          -- 42
Just (+) <*> Just 11 <*> Just 31  -- Just 42
```

Examples

```
(+) <$> Nothing <*> Just 2      -- Nothing
(+) <$> Just 1 <*> Just 2       -- Just 3

(,) <$> [1,2] <*> [6,7]        -- [(1,6),(1,7),(2,6),(2,7)]

-- intuition: all possible ways of multiplying the two input lists
pure (*) <*> [1,2] <*> [3,4]    -- [3,4,6,8]

-- in the context of lists, pure (*) = [(*)]
[(*), (+)] <*> [1,2] <*> [3,4]  -- [3,4,6,8,4,5,5,6]
```

Instances

instance Applicative Maybe where

```
pure x = Just x
Nothing <*> _ = Nothing
(Just f) <*> mx = fmap f mx
```

instance Applicative IO where

```
pure = return
mf <*> mx = do
  f <- mf
  x <- mx
  return (f x)
```

2.6.6.3. Monad

So far, we have seen how functors and applicative functors help us to:

- apply pure functions to “effectful” arguments
- compose function application with multiple “effectful” arguments

What is missing is how to apply and compose “effectful” functions with “effectful” arguments.

Definition

class Applicative m => Monad m where

```
(>=) :: m a -> (a -> m b) -> m b
(>>) :: m a -> m b -> m b
return :: a -> m a
{-# MINIMAL (>=) #-}
```

Type Class Laws

- 1) Left identity
 - `return x >= f = f x`
- 2) Right identity
 - `mx >= return = mx`
- 3) Associativity
 - `(mx >= f) >= g = mx >= (\x -> (f x >= g))`

Bind

Function that binds an effectful value into an effectful function

```
(>=) :: m a -> (a -> m b) -> m b
```

Kleisli arrow (not in haskell)

The Kleisli arrow (`<=>`) is analogous to normal function composition, except that it works on monadic functions

```
(.) :: (b -> c) -> (a -> b) -> a -> c
(<=>) :: Monad m => (b -> m c) -> (a -> m b) -> a -> m c
(f <=> g) x = g x >= f
```

Examples

```
safediv 8 0 >= flip safediv 2      -- Nothing
safediv 8 2 >= flip safediv 2      -- Just 2
```

```

safediv :: Int → Int → Maybe Int
safediv n m = if m == 0 then Nothing else Just $ div n m

do {x ← [1,2]; y ← [3,4]; return (x,y)}  -- [(1,3),(1,4),(2,3),(2,4)]
-- ⇔
[1,2] >=> \x → [3,4] >=> \y → return (x,y)
-- ⇔
[1,2] >=> \x → [3,4] >=> return . (x,)
-- ⇔
[1,2] >=> \x → (x,) <$> [3,4]
-- ⇔
(,) <$> [1,2] <*> [3,4]

```

Instances

```

instance Monad Maybe where
    Nothing >=> _ = Nothing
    (Just x) >=> f = f x

instance Monad [] where
    xs >=> f = [y | x ← xs, y ← f x]

```

2.6.6.4. Comparison

	<i>Application operator</i>	<i>Type of application operator</i>	<i>Function</i>	<i>Argument</i>	<i>Result</i>
<i>Pure function application</i>	juxtapose, ($\$$)	$(a \rightarrow b) \rightarrow a \rightarrow b$	pure	pure	pure
<i>Functor</i>	fmap, ($\langle \$ \rangle$)	$(a \rightarrow b) \rightarrow f\ a \rightarrow f\ b$	pure	effectual	effectual
<i>Applicative functor</i>	($\langle * \rangle$)	$f\ (a \rightarrow b) \rightarrow f\ a \rightarrow f\ b$	pure	effectual	effectual
<i>Monad</i>	($\rangle =$)	$m\ a \rightarrow (a \rightarrow m\ b) \rightarrow m\ b$	effectual	effectual	effectual

2.7. MODULES

- A collection of related values, types, classes, etc.
- Name: Identifier that starts with a capital letter.
- Naming convention for hierarchy: separated using "." (e.g. `Data.List`, `Data.List.NonEmpty`, `Data.Set`, ...)
- Each module `Dir.Name` must be contained in a file `Dir/Name.hs`.

```

module BinarySearchTree
( T, toList
) where

data T a = Leaf | Node (T a) a (T a)
    deriving Show

```

```
toList :: T a → [a]
```

```
-- ...
```

```
import qualified BinarySearchTree
  as Set (T, toList)
```

2.8. LAZY EVALUATION

- Perform an evaluation step only when it is necessary.
- Never perform the same step twice.

2.9. SIDE EFFECTS

Interactive programs can be written in Haskell by using types to distinguish pure expressions from impure actions that may involve side effects.

IO a -- The type of actions that return a value of type a

One may also use a let statement within a do block to perform a pure computation. Note that the let statement here is [different](#) from the let ... in ... statement for expressions.

```
act :: IO (Char, Char)
act = do {x ← getChar;
         y ← getChar;
         let {p = (x,y)};
         return p}
-- ⇔
```

```
act = do
  x ← getChar
  y ← getChar
  let p = (x,y)
  return p
```

Composing IO actions

```
(>=) :: Monad m ⇒ m a → (a → m b) → m b
```

```
do {x ← getChar; putChar x}
-- ⇔
getChar >= putChar
```

3. LAMBDA CALCULUS

Created by Alonzo Church.

3.1. SYNTAX

```
<M>      ::= <id> | λ<id>. <M> | <M> <M> | ( <M> )
<id>     ::= a | b | c | ... | 0 | 1 | 2 | ... | + | - | * | ...
```

Reserved words: “ λ ”, “.”, “(”, “)”

$$\begin{array}{c} \Leftrightarrow \\ \underbrace{M}_{\lambda\text{-term}} ::= \underbrace{x}_{\text{Id}} \mid \underbrace{\lambda x. M}_{\lambda\text{ abstraction}} \mid \underbrace{M M}_{\text{Application}} \\ \Leftrightarrow \end{array}$$

```
type Id = String
data Term
  = Var Id
  | Abs Id Term
  | App Term Term
```

Rules:

- The scope of a λ abstraction extends as far right as possible: $\lambda x. x \lambda y. y x \Leftrightarrow (\lambda x. (x (\lambda y. (y x))))$
- Application is left associative: $M_1 M_2 M_3 \Leftrightarrow (M_1 M_2) M_3$

3.2. DEFINITIONS

Bound variables: Bound variables are placeholders that refer to their binding occurrences. Their names have no significance and can be renamed (α -conversion).

$$\lambda \underbrace{x}_{\substack{\text{binding} \\ \text{occurrence}}} . \underbrace{x}_{\text{bound}} \underbrace{z}_{\text{free}}$$

Free variables: Variables that aren't bound

Open: A lambda term **with at least one free variable** is said to be **open**

Closed: A lambda term **without any free variables** is said to be **closed**

Combinators: Closed lambda terms are also called **combinators**, since all they do is combine their parameters in different ways

Closure: A lambda term whose free variables are bound by its enclosing scope

(β) Normal form: A λ term is said to be in **normal form** if **no further reductions** can be applied to it

Confluence: Every λ term has **at most one normal form**. This property is sometimes referred to the **confluence property** of β reduction

3.2.1. nfin

“Not Free In”

```
nfin :: Id → Term → Bool
x `nfin` (Var y) = x /= y
x `nfin` (Abs y m) = x == y || nfin x m
x `nfin` (App m n) = x `nfin` m && x `nfin` n
```

3.2.2. substitution

The term $[x := L]M$ denotes the term M with all free occurrences of the variable x replaced by the term L .

-- NOTE: renameBoundVars does what it sounds like

```
type Subst = (Id, Term)
applySubst :: Subst → Term → Term
applySubst (x, l) (Var y) = if x == y then l else Var y
applySubst (x, l) (Abs y m)
```

```

| x = y                                = Abs y m
| x ≠ y && y `nfin` l                 = Abs y $ applySubst (x, l) m
| otherwise
=
    applySubst (x, l) $ alphaConvert (Abs y m) newId -- newId `nfin` m
applySubst s (App m n)                = App (applySubst s m) (applySubst s n)

```

3.2.3. α conversion

$$\lambda x. M = \lambda y. \underbrace{[x := y] M}_{\text{substitution}} \text{ if } \underbrace{(y \text{ nfin } \lambda x. M)}_{\text{needed to avoid variable capture}}$$

```

alphaConvert :: Term → Id → Term
alphaConvert (Abs x m) y =
    if y `nfin` m
    then Abs y $ applySubst (x, Var y) m
    else Abs x m
alphaConvert x _ = x

```

Example 1:

$$\begin{aligned}
 & \lambda \underline{x}. \underline{x} z \\
 &= \lambda \underline{b}. \underline{b} z \vdash \alpha \\
 &= \lambda n. n z \vdash \alpha
 \end{aligned}$$

3.2.4. β reduction

Replacing formal parameters (placeholders) with actual parameters (values)

$$(\lambda x. M) N = [x := N]M$$

```

betaReduce :: Term → Maybe Term
betaReduce (App (Abs x m) n) = applySubst (x, n) m
betaReduce = id

```

Example 2:

$$\begin{aligned}
 & \underline{((\lambda x. (\lambda y. y)) ((\lambda x. x x) (\lambda x. x))) ((\lambda x. x) z)} \\
 &= \underline{(\lambda y. y) ((\lambda x. x) z)} \quad \vdash \beta \\
 &= \underline{(\lambda x. x) z} \quad \vdash \beta \\
 &= z \quad \vdash \beta
 \end{aligned}$$

3.2.5. δ reduction

Although not strictly necessary, it is very useful to introduce abbreviations for λ terms to make them more readable. This can be done by introducing definitions as equalities of the following form to a context within which a term can be reduced.

$$x = M$$

The substitution of a defined symbol with its definition is referred as δ reduction

Example 3:

$$\begin{aligned}
& \underline{\top} \top \perp = \top \\
\Leftrightarrow & \underline{(\lambda p. \lambda q. p p q)} \top \perp = \top \quad \because \delta \\
\Leftrightarrow & \underline{(\lambda q. \top \top q)} \perp = \top \quad \because \beta \\
\Leftrightarrow & \underline{\top} \top \perp = \top \quad \because \beta \\
\Leftrightarrow & \underline{(\lambda x. \lambda y. x)} \top \perp = \top \quad \because \delta \\
\Leftrightarrow & \underline{(\lambda y. \top)} \perp = \top \quad \because \beta \\
\Leftrightarrow & \top = \top \quad \because \beta
\end{aligned}$$

3.2.6. η contraction

For any f that does not contain any free occurrences of x the following equality holds:

$$\begin{aligned}
(\lambda x. \rightarrow f x) &= f \\
\lambda x. \rightarrow f (g x) &= f . g
\end{aligned}$$

$$\begin{aligned}
\lambda x. f x &\Leftrightarrow f \\
\lambda x. f (g x) &\Leftrightarrow f g
\end{aligned}$$

3.3. EVALUATION

Innermost Redex: Every redex not containing any other redex. A term can contain several innermost redexes

Outermost Redex: Every redex not contained in any other redex A term can contain several outermost redexes

Leftmost Innermost Redex: The leftmost redex not containing any other redex. A term can contain at most one leftmost innermost redex

Leftmost Outermost Redex: The leftmost redex not contained in any other redex. A term can contain at most one leftmost outermost redex

Leftmost Innermost Strategy: The leftmost innermost redex is reduced in each step

Leftmost Outermost Strategy: The leftmost outermost redex is reduced in each step

3.3.1. Relation to parameter passing

Call by value: Leftmost innermost reduction except that no reductions are performed within λ abstractions.

Call by name: leftmost outermost reduction except that no reductions are performed within λ abstractions.

Lazy evaluation: Used in order to make call by name more efficient. Terms that are duplicated during function application are shared such that they are evaluated at most once.

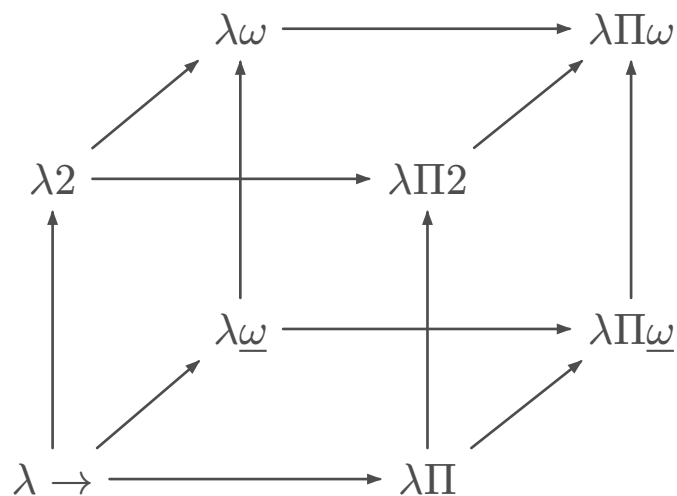
3.4. ENCODING DATA AND OPERATIONS

Term	Encoding
True	$\top = \lambda x. y x$
False	$\perp = \lambda x. y y$

Term	Encoding
Conjunction	$\wedge = \lambda p. \lambda q. p \ q \ p$
Disjunction	$\vee = \lambda p. \lambda q. p \ p \ q$
Negation	$\neg = \lambda p. p \ \perp \ \top$
Zero	$0 = \lambda f. \lambda x. x$
One	$\lambda f. \lambda x. f \ x$
Two	$\lambda f. \lambda x. f \ (f \ x)$
Successor	$\text{succ} = \lambda n. \lambda f. \lambda x. f \ (n \ f \ x)$
Addition	$+= \lambda m. \lambda n. m \ \text{succ} \ n$
Multiplication	$* = \lambda m. \lambda n. m \ (+ \ n) \ 0$
Power	$\text{power} = \lambda b. \lambda e. e \ b$
Predecessor	$\text{pred} = \lambda n. \lambda f. \lambda x. n \ (\lambda g. \lambda h. h \ (g \ f)) \ (\lambda u. x) \ (\lambda u. u)$
Minus	$- = \lambda m. \lambda n. n \ \text{pred} \ m$
Is zero	$\text{isZero} = \lambda n. n \ (\lambda x. \perp) \ \top$
Less than or equal	$\leq = \lambda m. \lambda n. \text{isZero} \ (- \ m \ n)$
Are equal	$\text{areEqual} = \lambda m. \lambda n. (\wedge \ (\leq \ m \ n) \ (\leq \ m \ n))$
Pair	$\text{pair} = \lambda x. \lambda y. \lambda f. f \ x \ y$
First	$\text{fst} = \lambda p. p \ \top$
Second	$\text{snd} = \lambda p. p \ \perp$

3.5. LAMBDA CUBE

https://www.cs.uoregon.edu/research/summerschool/summer23/_lectures/oplss-lambda-cube1.pdf



Origin: $\lambda \rightarrow$: Simply typed lambda calculus Terms may only depend on Terms Curry-Howard correspondence for $\lambda \rightarrow$: Propositional calculus restricted to only use implication.

Going up (2): $\lambda 2$: System F, second-order lambda calculus Terms may depend on Types (polymorphism, e.g. (Church-style) $\lambda \alpha : *. \lambda x : \alpha. x : \forall \alpha. (\alpha \rightarrow \alpha)$, or (Curry-style) $\lambda x.x : \forall \alpha. (\alpha \rightarrow \alpha)$) Curry-Howard correspondence for $\lambda 2$: fragment of second-order intuitionistic logic that uses only universal quantification.

Going inwards (ω): Types may depend on Types (type operators, e.g. **List** a is a type, where List is a type operator with kind $* \rightarrow *$) Not very interesting in isolation. Normally combined with $\lambda 2$ (System F) to give $\lambda \omega$ (System F ω) (a variant of this (System FC) is used in Haskell) Curry-Howard correspondence for $\lambda \omega$ (System F ω): Higher-Order Logic.

Going rightwards (Π , or P): Types may depend on values (dependent types, e.g. **FloatList 3** is a type denoting a list of floats with length 3, where **FloatList** : **Nat** $\rightarrow *$) $\lambda \Pi$: also called λP , LF Curry-Howard correspondence for $\lambda \Pi$: A form of predicate calculus that only uses implication and universal quantification.

Richest calculus of all 8: $\lambda \Pi \omega$: Calculus of Constructions (CC, CoC, λC)

4. PROOFS

PAT: Propositions As Types / Proofs As Terms

4.1. NOTATION

4.1.1. Symbols

Term	Definition
$\wedge$	And
$\vee$	Or
$\neg$	Not
$\rightarrow$	Implies ($\Rightarrow$)
$\leftrightarrow$	Biconditional (iff/ $\Leftrightarrow/\equiv$)
$\top$	Tautology (always true)
$\perp$	Contradiction (always false)
$\forall$	Universal quantifier (for all)
$\exists$	Existential quantifier (exists)
$\exists!$	There exists exactly one
$\nexists$	There does not exist
$\models$	Logical consequence (entails)
$\vdash$	Provability (can be proven from)

4.1.2. Proof tree

$$\frac{\text{Premise 1} \quad \text{Premise 2} \quad \dots \quad \text{Premise n}}{\text{Conclusion}} \text{Rule name}$$

Example 4: $\lambda \rightarrow$

$$\frac{}{\Gamma, x : \sigma \vdash x : \sigma} \text{var} \quad \frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash MN : \tau} \text{app}_{\text{term}} \quad \frac{\Gamma, x : \sigma \vdash M : \tau}{\Gamma \vdash \lambda x. M : \sigma \rightarrow \tau} \text{abs}_{\text{term}}$$

Example 5: Type inference

$$\frac{f :: A \rightarrow B \quad e :: A}{f \ e :: B}$$

4.1.3. Two-Column Proof

Format:

Statement	Justification
-----------	---------------

Example 6:

1. $P \rightarrow Q$	Premise
2. P	Premise
3. Q	From 1, 2 by Modus Ponens

4.1.4. Structured Proof

Format:

Theorem: [Statement to prove]

Proof:

1. [First statement with justification]
 2. [Second statement with justification]
 - ...
 - n. [Final conclusion]
- QED (or $\square$)

Example 7:

Theorem: If P and $P \rightarrow Q$, then Q .

Proof:

1. Assume P and $P \rightarrow Q$ as premises. (Assumption)
2. From $P \rightarrow Q$ and P , we conclude Q . (Modus Ponens)

$\square$

4.1.5. Sequent calculus

Format:

$$\Gamma \vdash \Delta$$

Where:

- Γ = set of premises (what we assume)
- $\vdash$ = “proves” or “entails”

– Δ = conclusion(s)

Example 8:

$$P \rightarrow Q, P \vdash Q$$

4.2. FORMAL LOGIC

Propositional logic: The simplest form of formal logic, dealing with statements (called propositions).

Proposition: Can be either true ($\top$) or false ($\perp$)

First-order logic/Predicate logic: Extension of propositional logic. It introduces variables, quantifiers, and predicates to express more nuanced statements about objects and their relationships.

Premise: Statement or assumption that you accept as true as the starting point for reasoning. Premises are the givens in a logical argument.

Sound reasoning: Logical argument is both valid and based on true premises

4.2.1. Formula

A formula is a symbolic expression built from logical operators, variables, quantifiers, and predicates according to strict grammatical rules. It's the written representation of a logical statement.

4.2.1.1. Components of a formula

Component	Meaning	Example
Propositional variables	Simple true/false statements	P, Q, R
Predicates	Properties or relationships	$\text{Human}(x), \text{Loves}(x, y)$
Quantifiers	Universal or existential claims	$\forall x, \exists y$
Logical operators	Combine or modify statements	$\wedge, \vee, \neg, \rightarrow, \leftrightarrow$
Parentheses	Clarify structure and precedence	$(P \wedge Q) \rightarrow R$

4.2.2. Proof

A proof is a sequence of logical steps that demonstrates the truth of a statement (called the theorem or conclusion) based on accepted premises and logical rules. Each step must follow necessarily from previous steps or established axioms.

4.2.2.1. Components of a proof

Component	Meaning	Example
Axioms/Premises	Starting assumptions accepted as true	All humans are mortal
Logical rules	Valid inference methods (e.g., modus ponens)	If $P \rightarrow Q$ and P is true, then Q is true
Intermediate steps	Conclusions derived from previous steps	Socrates is human, so Socrates is mortal
Final conclusion	The statement being proven	Socrates is mortal

4.2.2.2. Common proof techniques

- Direct proof
- Proof by contradiction

– Proof by [“Induction”](#) ([Page 22](#))

4.2.2.3. Logical rules

A **Rule of inference** is a logical principle that allows you to derive a new conclusion from one or more premises.

4.2.2.3.1. Modus ponens

$$\frac{P \rightarrow Q \quad P}{Q}$$

Example 9:

$$\frac{\text{If FP is based, then Haskell is based} \quad \text{FP is based}}{\text{Therefore, Haskell is based}}$$

4.2.2.3.2. Modus tollens

$$\frac{P \rightarrow Q \quad \neg Q}{\neg P}$$

Example 10:

$$\frac{\text{If Java is cool, then cool people write java} \quad \text{Cool people don't write java}}{\text{Therefore, Java isn't cool}}$$

4.2.2.3.3. Hypothetical syllogism

$$\frac{P \rightarrow Q \quad Q \rightarrow R}{P \rightarrow R}$$

Example 11:

$$\frac{\begin{array}{l} \text{If you live with other people, you are part of a society} \\ \text{If you are part of a society, you must abide by its rules} \end{array}}{\text{Therefore, if you live with other people, you must abide by their rules}} \text{ Ragebait}$$

4.2.2.3.4. Disjunctive syllogism

$$\frac{P \vee Q \quad \neg P}{Q}$$

Example 12:

$$\frac{\text{I am alive or dead} \quad \text{I am alive}}{\text{Therefore, I am not dead}}$$

4.2.2.3.5. Double negation elimination

$$\frac{\neg \neg P}{P}$$

Example 13:

We were not unable to meet the deadline
We were able to meet the deadline

4.3. EQUATIONAL REASONING

In this course the '==' symbol is used instead of '=' to be more consistent with the notation of equality used in Haskell.

Example 14:

```

    ∀x. add Zero x == x                                (addZero)
    ∀x. x == x                                           ((=)refl)

    add Zero (add y z) == add (add Zero y) z
  == { applying addZero }
    add y z == add (add Zero y) z
  == { applying addZero }
    add y z == add y z
  == { applying ((=)refl) }
    True

```

4.4. INDUCTION

≈ recursion

Structure:

- Proposition (Required to prove – RTP)
- Proof
 - 1) Base case
 - 2) Induction step
- Conclusion

Example 15: length (xs ++ ys) == length xs + length ys**Given properties**

```

    length [] == 0                                       (length[])
    ∀x xs. length (x:xs) == 1 + length xs              (length(:))
    ∀xs. [] ++ xs == xs                                ((++)[])
    ∀x xs ys. (x:xs) ++ ys == x:(xs++ys)              ((++)(:))
    ∀n. 0 + n == n                                       ((+)0)
    ∀a b c. (a + b) + c == a + (b + c)                 ((+)assoc)

```

$$\forall x. (x = x) = \text{True} \quad ((=)_{\text{refl}})$$

Required to Prove (RTP)

$$\forall xs \ ys. (\text{length } (xs ++ ys) = \text{length } xs + \text{length } ys)$$

Proof

Proceed by induction on xs:

Let $P \ xs = \forall ys. (\text{length } (xs ++ ys) = \text{length } xs + \text{length } ys)$ and apply the induction rule for lists on $P \ xs$.

1) Base Case. RTP: $P \ []$

$$\begin{aligned} &P \ [] \\ &= \{\text{applying definition of } P, \text{ choosing a fixed but arbitrary } ys\} \\ &\quad \text{length } ([] ++ ys) = \text{length } [] + \text{length } ys \\ &= \{\text{applying length}_{[]} \} \\ &\quad \text{length } ([] ++ ys) = 0 + \text{length } ys \\ &= \{\text{applying } (++)_{[]} \} \\ &\quad \text{length } ys = 0 + \text{length } ys \\ &= \{\text{applying } (+)_0 \} \\ &\quad \text{length } ys = \text{length } ys \\ &= \{\text{applying } (=)_{\text{refl}} \} \\ &\text{True} \end{aligned}$$

2) Induction Step. RTP: $\forall x \ xs. (P \ xs \Rightarrow P \ (x:xs))$

Choose a fixed but arbitrary x and xs , and assume that following the induction hypothesis $P \ xs$ holds

$$\forall ys. (\text{length } (xs ++ ys) = \text{length } xs + \text{length } ys) \quad (\text{Induction Hypothesis})$$

$$\begin{aligned} &P \ (x:xs) \\ &= \{\text{applying definition of } P, \text{ choosing a fixed but arbitrary } ys\} \\ &\quad \text{length } ((x:xs) ++ ys) = \text{length } (x:xs) + \text{length } ys \\ &= \{\text{applying length}_{(:)} \} \\ &\quad \text{length } ((x:xs) ++ ys) = (1 + \text{length } xs) + \text{length } ys \\ &= \{\text{applying } (++)_{(:)} \} \\ &\quad \text{length } (x:(xs ++ ys)) = (1 + \text{length } xs) + \text{length } ys \\ &= \{\text{applying length}_{(:)} \} \\ &\quad 1 + \text{length } (xs ++ ys) = (1 + \text{length } xs) + \text{length } ys \\ &= \{\text{applying Induction Hypothesis}\} \\ &\quad 1 + (\text{length } xs + \text{length } ys) = (1 + \text{length } xs) + \text{length } ys \\ &= \{\text{applying } (+)_{\text{assoc}} \} \\ &\quad 1 + (\text{length } xs + \text{length } ys) = 1 + (\text{length } xs + \text{length } ys) \\ &= \{\text{applying } (=)_{\text{refl}} \} \\ &\text{True} \end{aligned}$$

Conclusion

Concatenating two lists results in a list of length equal to the sum of the concatenated lists

4.4.1. Logical inference

Logical inference rule / proof rule syntax:

$$\frac{[\text{Base Case}] \quad \overbrace{([\text{Induction Hypothesis}] \Rightarrow [\text{Induction Step}])}^{\text{Inductive Case}}}{[\text{Main goal to be proven}]}$$

Example 16: $\sum_{k=0}^n k = \frac{k(k+1)}{2}$

- Base case: $0 = \frac{0(0+1)}{2}$
- Induction step: Show that for every $k \geq 0$, if $P(k)$ holds, then $P(k+1)$ also holds

$$\begin{aligned} \frac{k(k+1)}{2} + (k+1) &= \frac{k(k+1) + 2(k+1)}{2} \\ &= \frac{(k+1)(k+2)}{2} \\ &= \frac{(k+1)((k+1)+1)}{2} = \sum_{k=0}^n k + (k+1) \end{aligned}$$

Logical inference rule:

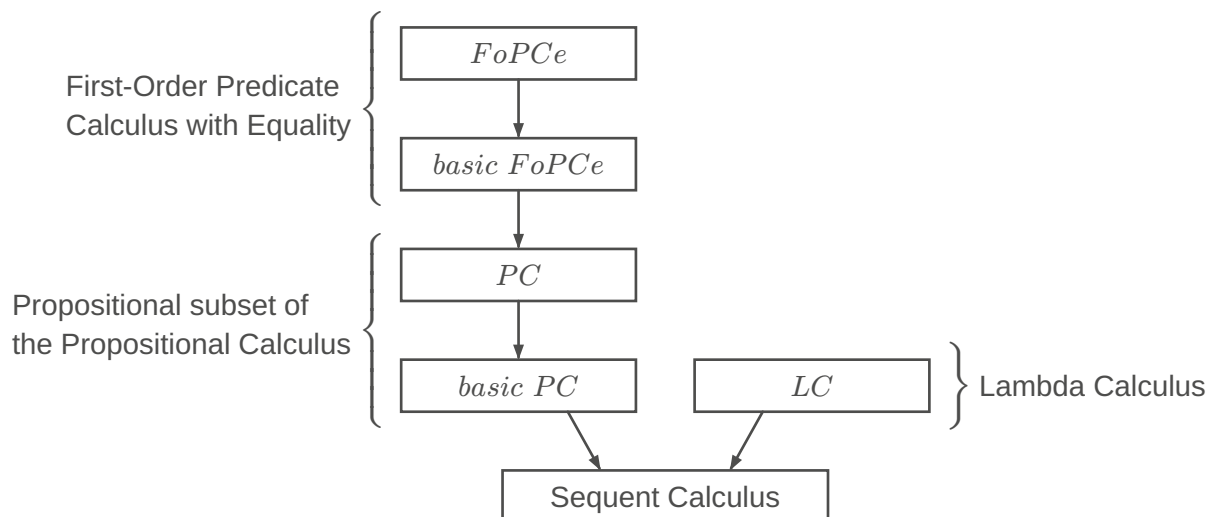
$$\frac{P0 \quad \forall n. (Pn \Rightarrow P(n+1))}{\forall n. (Pn)}$$

$$Pn = \left(\sum_{k=0}^n k = \frac{k(k+1)}{2} \right)$$

Example 17: `data [a] = [] | a:[a]`

$$\frac{P [] \quad \forall x \, xs. (P \, xs \Rightarrow P(x:xs))}{\forall xs. (P \, xs)}$$

4.5. THEORIES

**TODO:**

- Dependent typing
- Mutable state + parallel programming
- denotative language / semantics (central passages)

5. SOME CURSED TYPESCRIPT IG

```
type And<A extends boolean, B extends boolean> = A extends true ? B : false;
type Or<A extends boolean, B extends boolean> = A extends true ? true : B;
```

```
type Num = { prev?: Num };
type Zero = Num & { prev: undefined };
```

```
type Next<N extends Num> = Num & { prev: N };
type Prev<N extends Num> = N extends Zero ? Zero : Num & N["prev"];
```

```
type One = Next<Zero>;
type Two = Next<One>;
type Three = Next<Two>;
```

```
type Add<A extends Num, B extends Num> = A extends Zero
  ? B
  : A extends Zero
    ? B
    : Add<Prev<A>, Next<B>>;
```

```
type Sub<A extends Num, B extends Num> = B extends Zero
  ? A
  : A extends Zero
    ? Zero
    : Sub<Prev<A>, Prev<B>>;
```

```
type ToTuple<N extends Num, Acc extends any[] = []> = N extends {
  prev: infer P extends Num;
}
  ? ToTuple<P, [...Acc, "_"]>
  : Acc;
```

```
type ToLiteral<N extends Num> = ToTuple<N>["length"];
```

```
type ToLiteralTest = ToLiteral<Three>; // 3
type AddTest = ToLiteral<Add<Three, Two>>; // 5
type SubTest = ToLiteral<Sub<Three, Two>>; // 1
```